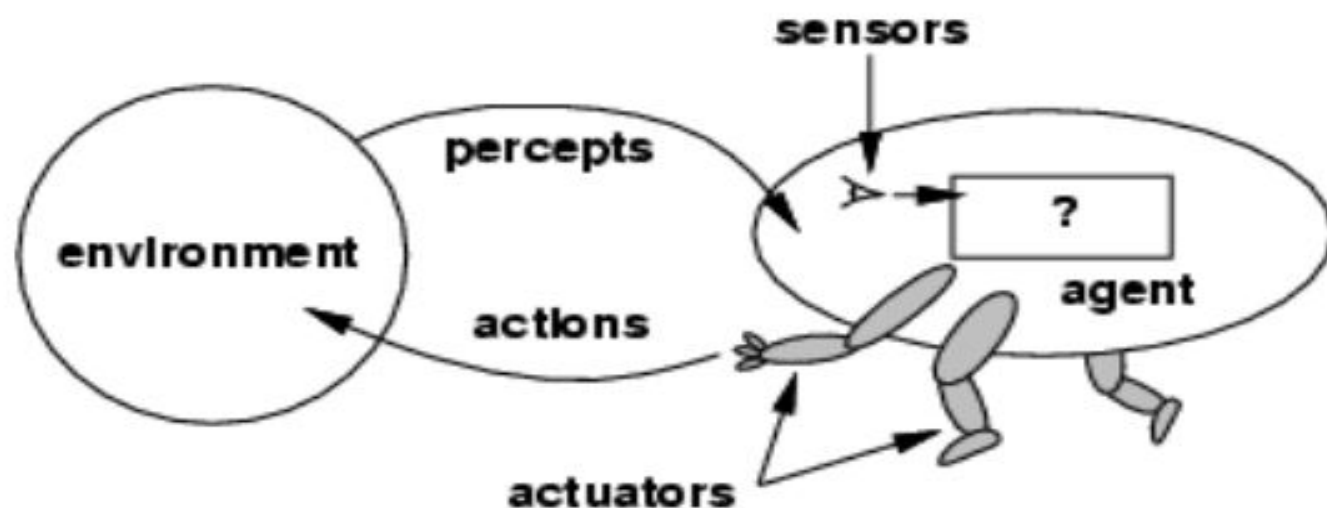


Agents

Agents

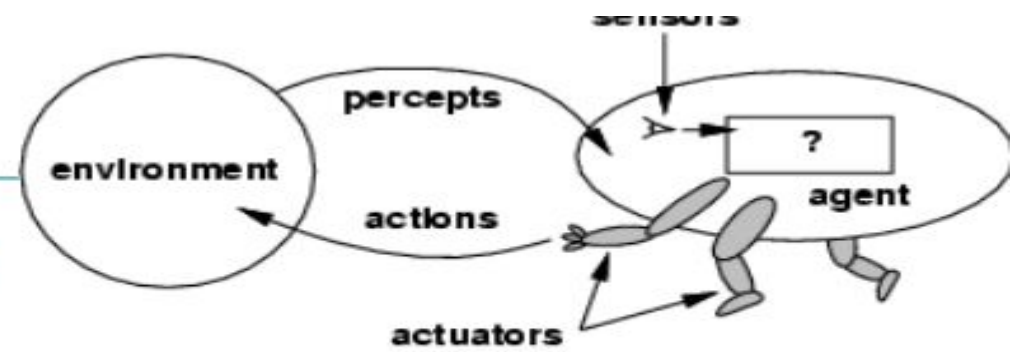
- An **agent** is anything that can be viewed as
 - **perceiving** its **environment** through **sensors** and
 - **acting** upon that environment through **actuators**
 - Assumption: Every agent can perceive its own actions (but not always the effects)



Agents

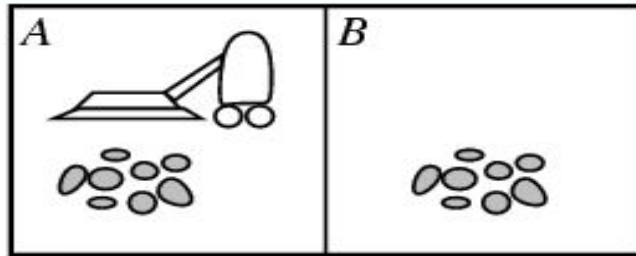
- Human agent:
 - eyes, ears, and other organs for sensors;
 - hands, legs, mouth, and other body parts for actuators
- Robotic agent:
 - cameras and infrared range finders for sensors;
 - various motors for actuators
- A software agent:
 - Keystrokes, file contents, received network packages as sensors
 - Displays on the screen, files, sent network packets as actuators

Agents and environments



- **Percept**: agent's perceptual input at any given instant
- **Percept sequence**: complete history of everything the agent has ever perceived
- An agent's choice of action at any given instant can depend on the entire percept sequence observed to date
- An agent's behavior is described by the **agent function** which maps from percept histories to actions:
$$[f: P^* \rightarrow A]$$
- We can imagine tabulating the agent function that describes any given agent (External characterization)
- Internally, the agent function will be implemented by an **agent program** which runs on the physical **architecture** to produce f
- agent = architecture + program

Vacuum-cleaner world



- Two locations: A and B
- Percepts: location and contents, e.g., [A,Dirty]
- Actions: *Left, Right, Suck, NoOp*

Percept sequence	Actions
[A,Clean]	Right
[A, Dirty]	Suck
[B,Clean]	Left
[B,Dirty]	Suck
[A,Clean],[A,Clean]	Right
[A,Clean],[A,Dirty]	Suck
...	...
[A,Clean],[A.Clean],[A,Clean]	Right
[A,Clean],[A,Clean],[A,Clean]	Suck

One simple function is :

if the current square is dirty then suck, otherwise move to the other square

Rational agents

- An agent should strive to "do the right thing", based on what it can perceive and the actions it can perform.
- The right action is the one that will cause the agent to be most successful
- Performance measure: An objective criterion for success of an agent's behavior
- E.g., performance measure of a vacuum-cleaner agent could be amount of dirt cleaned up, amount of time taken, amount of electricity consumed, amount of noise generated, etc.
- As a general rule, it is better to design performance measures according to what one actually wants in the environment. Rather than according to how one thinks the agent should behave (amount of dirt cleaned vs a clean floor)
- A more suitable measure would reward the agent for having a clean floor

Rationality

- What is rational at any given time depends on four things
 - The performance measure that defines the criterion of success
 - The agent's prior knowledge of the environment
 - The actions that the agent can perform
 - The agent's percept sequence to date
- **Rational Agent:** For each possible percept sequence, a rational agent should select an action that is expected to maximize its performance measure, given the evidence provided by the percept sequence and whatever built-in knowledge the agent has.

Vacuum cleaner agent

- Let's assume the following
 - The performance measure awards one point for each clean square at each time step, over a lifetime of 1000 time steps
 - The geography of the environment is known a priori but the dirt distribution and the initial location of the agent are not. Clean squares stay clean and the sucking cleans the current square. The Left and Right actions move the agent left and right except when this would take the agent outside the environment, in which case the agent remains where it is
 - The only available actions are Left, Right, Suck and NoOp
 - The agent correctly perceives its location and whether that location contains dirt
- Under these circumstances the agent is rational: its expected performance is at least as high as any other agent's

Vacuum cleaner agent

- Same agent would be irrational under different circumstances
 - once all dirt is cleaned up it will oscillate needlessly back and forth.
 - If the performance measure includes a penalty of one point for each movement left or right, the agent will fare poorly.
 - A better agent for this case would do nothing once it is sure that all the squares are clean.
 - If the clean squares can become dirty again, the agent should occasionally check and clean them if needed.
 - If the geography of the environment is unknown the agent will need to explore it rather than stick to squares A and B

Rational agents

- Rationality is distinct from omniscience (all-knowing with infinite knowledge)
- Rationality maximizes expected performance while perfection maximizes actual performance
- Agents can perform actions in order to modify future percepts so as to obtain useful information (information gathering, exploration)
- An agent is **autonomous** if its behavior is determined by its own experience (with ability to learn and adapt)

Specifying the task environment (PEAS)

- PEAS:
 - Performance measure,
 - Environment,
 - Actuators,
 - Sensors
- In designing an agent, the first step must always be to specify the task environment (PEAS) as fully as possible

PEAS for an automated taxi driver

- **Performance measure:** Safe, fast, legal, comfortable trip, maximize profits
- **Environment:** Roads, other traffic, pedestrians, customers
- **Actuators:** Steering wheel, accelerator, brake, signal, horn
- **Sensors:** Cameras, sonar, speedometer, GPS, odometer, engine sensors, keyboard

Environment types

- Fully observable vs. partially observable
- Deterministic vs. stochastic
- Episodic vs. sequential
- Static vs. dynamic
- Discrete vs. continuous
- Single agent vs. multiagent

Environment types

Fully observable vs. partially observable:

- An environment is fully observable if an agent's sensors give it access to the complete state of the environment at each point in time.
- Fully observable environments are convenient, because the agent need not maintain any internal state to keep track of the world
- An environment might be partially observable because of noisy and inaccurate sensors or because parts of the state are simply missing from the sensor data
- Examples: vacuum cleaner with local dirt sensor, taxi driver

Environment types

Deterministic vs. stochastic:

- The environment is deterministic if the next state of the environment is completely determined by the current state and the action executed by the agent.
- In principle, an agent need not worry about uncertainty in a fully observable, deterministic environment
- If the environment is partially observable then it could appear to be stochastic
- Examples: Vacuum world is deterministic while taxi driver is not
- If the environment is deterministic except for the actions of other agents, then the environment is **strategic**

Environment types

Episodic vs. sequential:

- In episodic environments, the agent's experience is divided into atomic "episodes" (each episode consists of the agent perceiving and then performing a single action), and the choice of action in each episode depends only on the episode itself.
- Examples: classification tasks
- In sequential environments, the current decision could affect all future decisions
- Examples: chess and taxi driver

Environment types

Static vs. dynamic:

- The environment is unchanged while an agent is deliberating.
- Static environments are easy to deal with because the agent need not keep looking at the world while it is deciding on the action or need it worry about the passage of time
- Dynamic environments continuously ask the agent what it wants to do
- The environment is **semi-dynamic** if the environment itself does not change with the passage of time but the agent's performance score does
- Examples: taxi driving is dynamic, chess when played with a clock is semi-dynamic, crossword puzzles are static

Environment types

Discrete vs. continuous:

- A limited number of distinct, clearly defined states, percepts and actions.
- Examples: Chess has finite number of discrete states, and has discrete set of percepts and actions. Taxi driving has continuous states, and actions

Environment types

Single agent vs. multiagent:

- An agent operating by itself in an environment is single agent
- Examples: Crossword is a single agent while chess is two-agents
- Question: Does an agent A have to treat an object B as an agent or can it be treated as a stochastically behaving object
- Whether B's behaviour is best described by as maximizing a performance measure whose value depends on agent's A behaviour
- Examples: chess is a competitive multiagent environment while taxi driving is a partially cooperative multiagent environment

Environment types

Task Environment	Observable	Deterministic	Episodic	Static	Discrete	Agents
Crossword puzzle	Fully	Deterministic	Sequential	Static	Discrete	Single
Chess with a clock	Fully	Strategic	Sequential	Semi	Discrete	Multi
Poker	Partially	Stochastic	Sequential	Static	Discrete	Multi
Backgammon	Fully	Stochastic	Sequential	Static	Discrete	Multi
Taxi driving	Partially	Stochastic	Sequential	Dynamic	Continuous	Multi
Medical Diagnosis	Partially	Stochastic	Sequential	Dynamic	Continuous	Single
Image Analysis	Fully	Deterministic	Episodic	Semi	Continuous	Single
Part-picking robot	Partially	Stochastic	Episodic	Dynamic	Continuous	Single
Refinery controller	Partially	Stochastic	Sequential	Dynamic	Continuous	Single
Interactive English Tutor	Partially	Stochastic	Sequential	Dynamic	Discrete	Multi

- The environment type largely determines the agent design
- The real world is (of course) partially observable, stochastic, sequential, dynamic, continuous, multi-agent

Agent types

- Simple reflex agents
- Model-based reflex agents
- Goal-based agents
- Utility-based agents

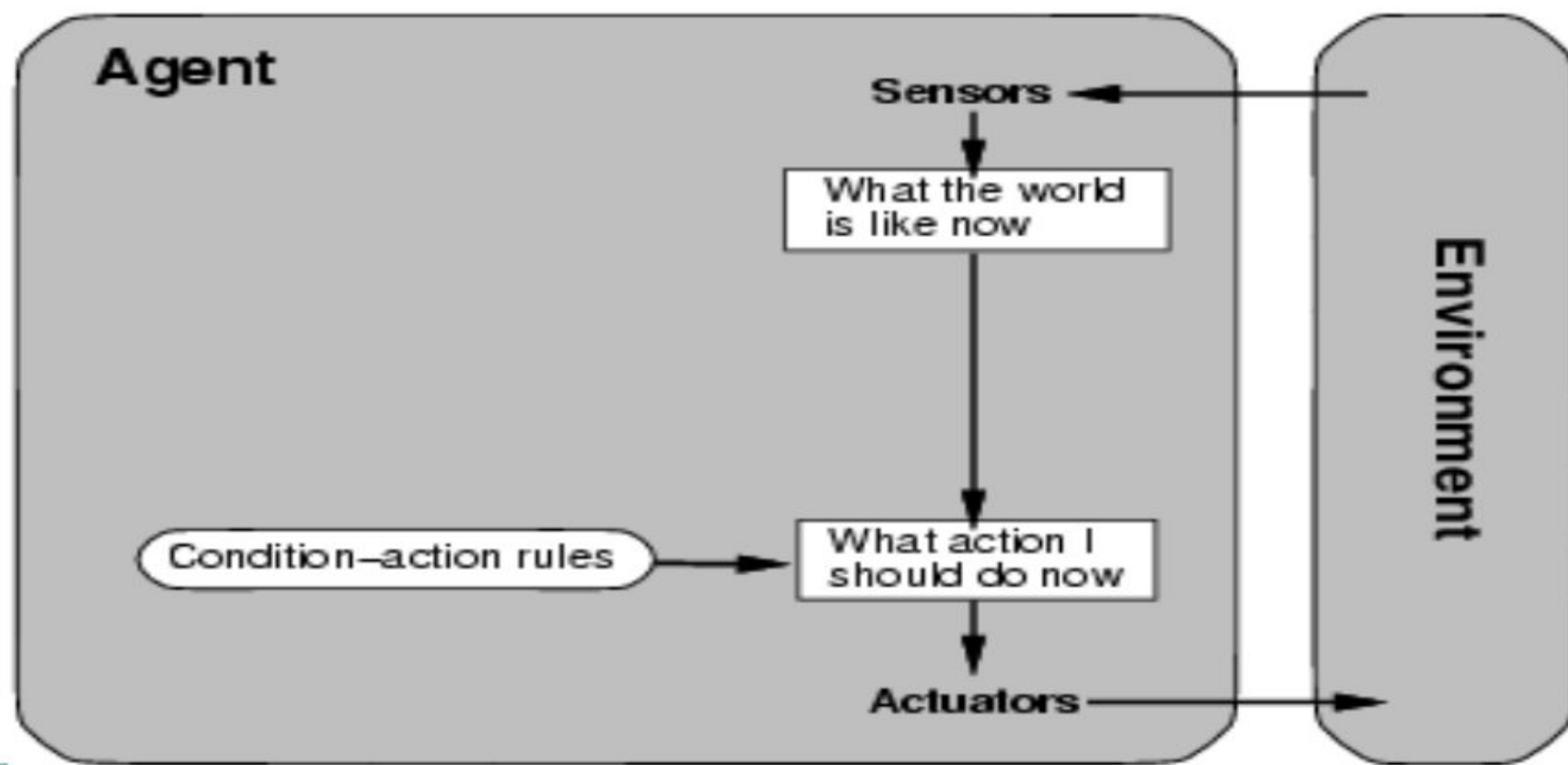
Simple reflex agents

- Select actions on the basis of the **current percept** ignoring the rest of the percept history
- Example: simple reflex vacuum cleaner agent

```
function REFLEX-VACUUM-AGENT([location,status]) returns an action  
  if status = Dirty then return Suck  
  else if location = A then return Right  
  else if location = B then return Left
```

- **Condition-action-rule**
- Example: **if** *car-in-front-is-breaking* **then** *initiate-breaking*

Simple reflex agents



Applies the brake



Condition-Action Rule:

These agents follow "if-then" rules, like "if someone is in front of the car then initiate braking."

Simple reflex agents

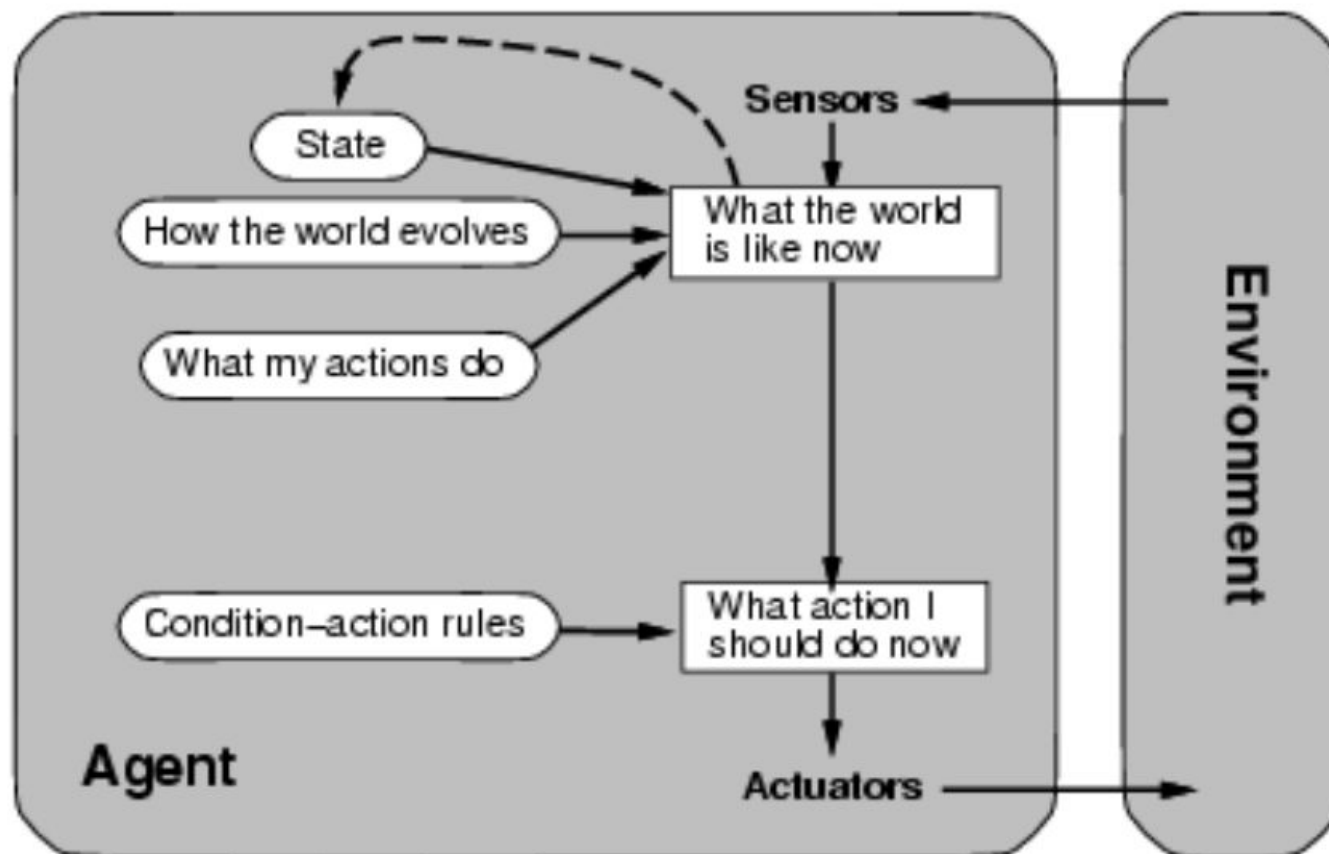
```
function SIMPLE-REFLEX-AGENT(percept) returns an action
  static: rules, a set of condition-action rules
  state <-- INTERPRET_INPUT(percept)
  rule <-- RULE_MATCH(state, rules)
  action <-- RULE_ACTION[rule]
  return action
```

- Simple-reflex agents are simple, but they turn out to be of very limited intelligence
- The agent will work only if the correct decision can be made on the basis of the current percept – that is only if the environment is fully observable
- Infinite loops are often unavoidable – escape could be possible by randomizing

Model-based reflex agents

- The agent should keep track of the part of the world it can't see now
- The agent should maintain some sort of internal state that depends on the percept history and reflects at least some of the unobserved aspects of the current state
- Updating the internal state information as time goes by requires two kinds of knowledge to be encoded in the agent program
 - Information about how the world evolves independently of the agent
 - Information about how the agent's own actions affects the world
- Model of the world – model based agents

Model-based reflex agents

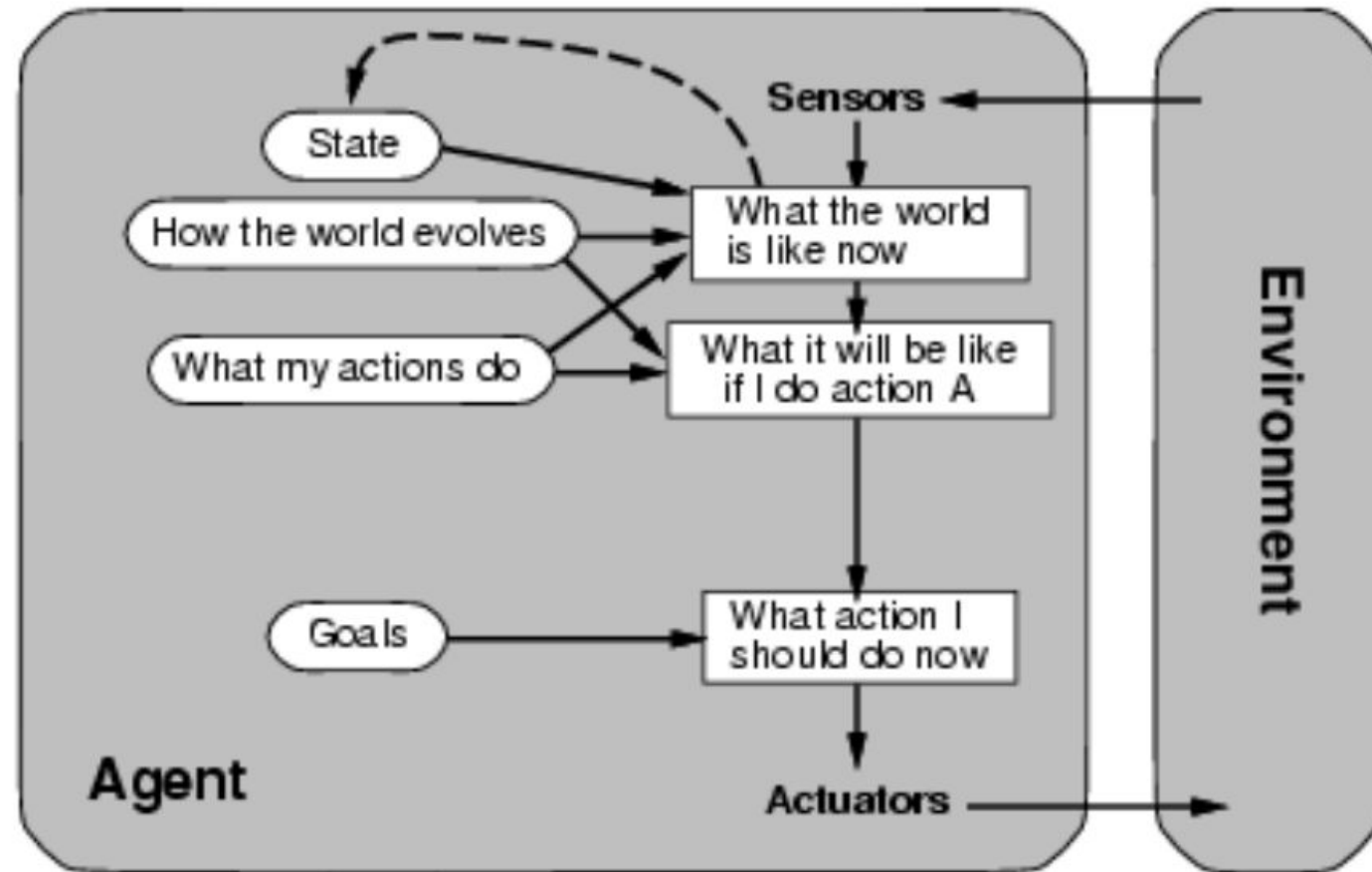


- Internal Model: The car has a map of the roads, traffic rules, and typical traffic patterns.
- Sensors: Uses cameras and sensors to perceive traffic lights, road signs, other vehicles, pedestrians, and road conditions.
- Updating Knowledge: It updates its model in real-time as it drives, learning current traffic conditions, construction work, or detours.
- Decision Making: If the car approaches a red light, it stops. When the light turns green and the path is clear, it proceeds.
- Learning Over Time: The car refines its internal model to improve predictions, like estimating how long it will take to reach destinations considering the current traffic.

Goal-based agents

- Knowing about the current state of the environment is not always enough to decide what to do (e.g. decision at a road junction)
- The agent needs some sort of goal information that describes situations that are desirable
- The agent program can combine this with information about the results of possible actions in order to choose actions that achieve the goal
- Usually requires search and planning

Goal-based agents



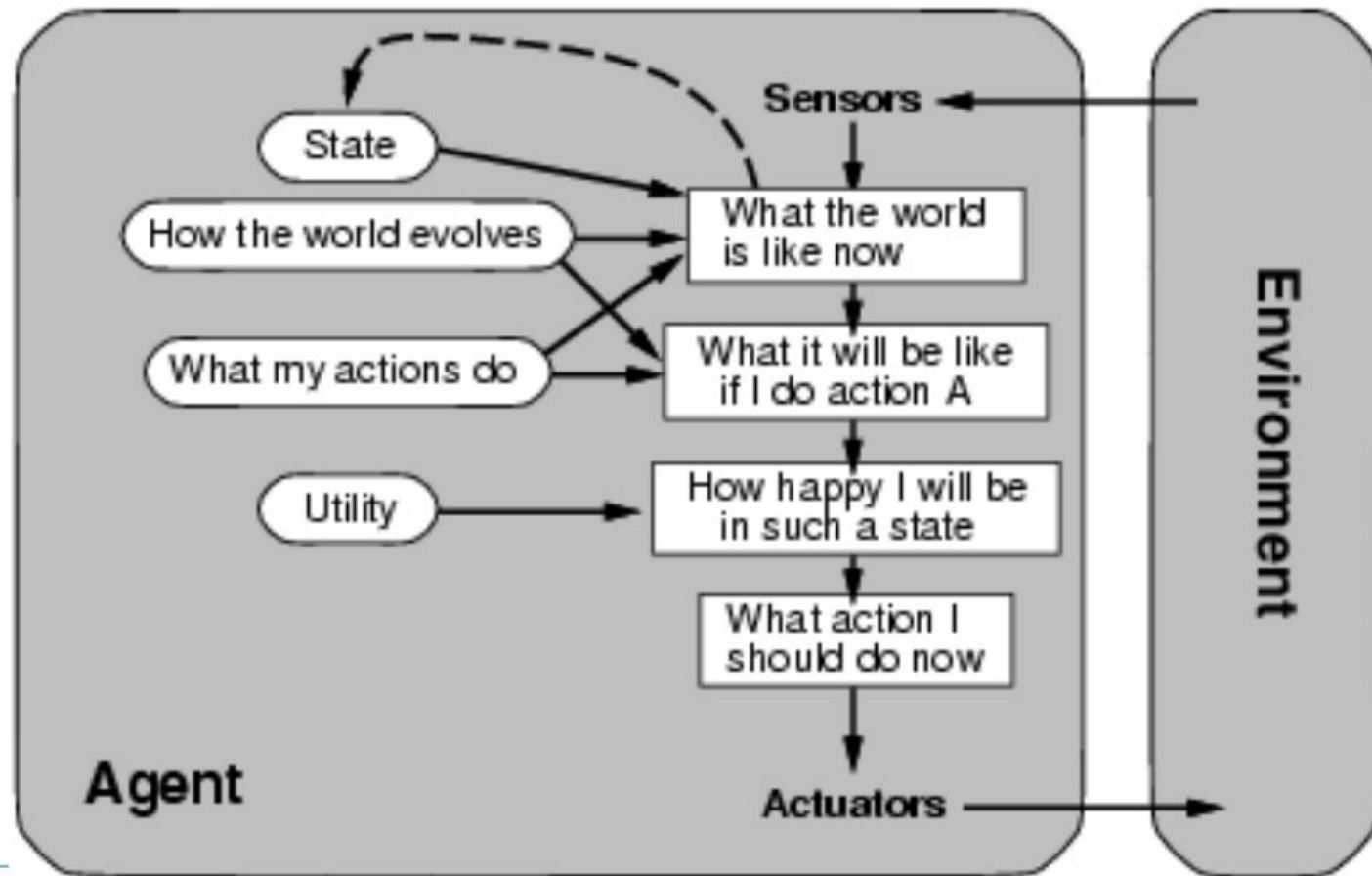
Goal-based agents vs reflex-based agents

- Although goal-based agents appears less efficient, it is more flexible because the knowledge that supports its decision is represented explicitly and can be modified
- On the other hand, for the reflex-agent, we would have to rewrite many condition-action rules
- The goal based agent's behavior can easily be changed
- The reflex agent's rules must be changed for a new situation

Utility-based agents

- Goals alone are not really enough to generate high quality behavior in most environments – they just provide a binary distinction between happy and unhappy states
- A more general performance measure should allow a comparison of different world states according to exactly how happy they would make the agent if they could be achieved
- Happy – Utility (the quality of being useful)
- A utility function maps a state onto a real number which describes the associated degree of happiness

Utility-based agents



Architecture for Intelligent Agents

- The architecture for intelligent agents broadly refers to the underlying structure or model that defines how the agents perceive their environment, make decisions, and take actions. There are several types of architectures, each with its own approach to processing and responding to stimuli:

Reactive Architecture

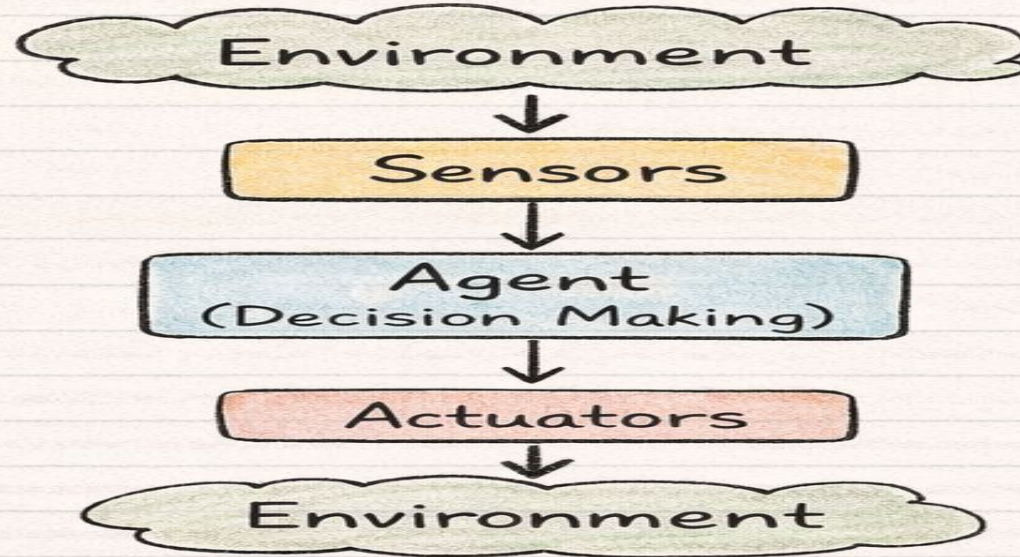
- **Reactive Architecture**
- Reactive architecture is a type of agent architecture where the agent:
- Responds **directly to current inputs**
- Does **not store past information**
- Has **no internal model**
-

Working

- Sensors detect environment
- Simple rules are applied
- Immediate action is taken
- No planning, only reaction

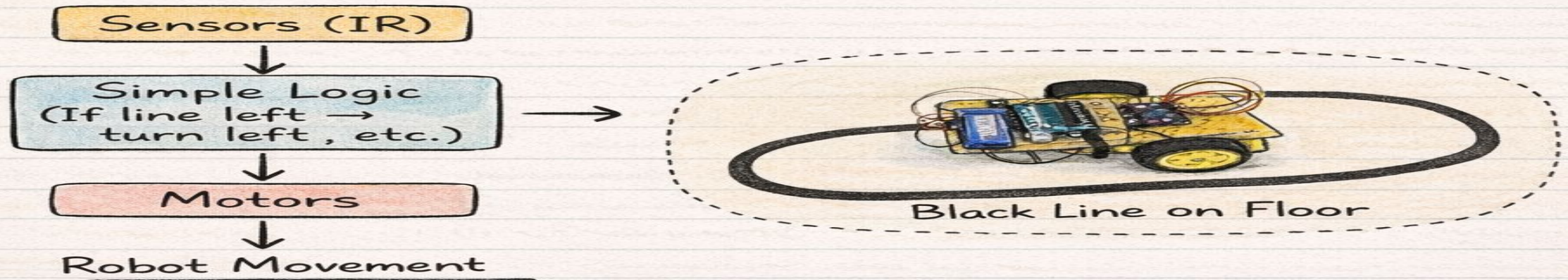
Example: A line-following robot that adjusts its path based on real-time sensor input without an internal representation of the environment.

Architecture for Intelligent Agents



Reactive Architecture:

Example: Line-Following Robot



Deliberative / Reasoning Architecture

Deliberative architecture is a type of agent architecture where the agent:

- Uses an **internal world model**
- Applies **planning and reasoning**
- Makes decisions based on **goals** . *Think first, then act*

◆ Features

- Maintains **internal representation** of environment
- Uses **logic and reasoning**
- Performs **planning before action**
- Goal-oriented behavior

Example

- **NASA's Mars rovers**
- Maintain a **map of surroundings**
- Use **planning algorithms**
- Perform tasks like:
 - Navigation
 - Experimentation



Hybrid Architecture

Hybrid architecture combines both:

- **Reactive architecture** (fast response)
- **Deliberative architecture** (planning & reasoning)

It uses the **best of both approaches**.

Example: Autonomous Vehicles

- **Reactive part:**
 - Avoids sudden obstacles
 - Prevents collisions
- **Deliberative part:**
 - Plans route
 - Navigates efficiently

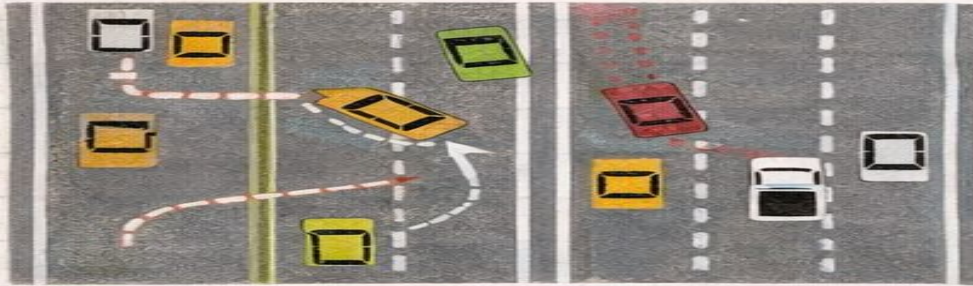
Hybrid Architecture



Key Idea: Think + React

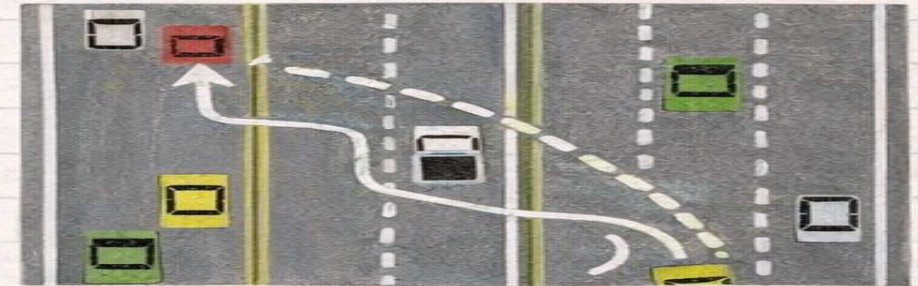
Example: Autonomous Vehicles (Reactive + Deliberative)

Reactive Part: Collision Avoidance



Quick Reaction,
Prevents Collision

Deliberative Part: Route Planning



Plans Efficient
Path

Reactive Part: Collision Avoidance



Quick Reaction,
Prevents Collision

(a) Collision Avoidance

Deliberative Part: Route Planning



(b) Reactive Part: Route Planning



Introduction to OpenAI Agents SDK,
LangGraph, CrewAI, AutoGen,